

Escuela Superior Politécnica de Chimborazo



Actividad autónoma colaborativa. Diseño arquitectónico de la aplicación

Estudiantes:

Kevin Gerardo Quilligana Vivas

Liliana Berzabe Paguay Carrillo

Facultad de Informática y Electrónica, Ingeniería de Software

Aplicaciones Informáticas II

Riobamba, 21 de abril del 2026

1 Introducción y Objetivos

1.1 Requisitos y Objetivos

Elemento	Descripción
Objetivo de Negocio	Automatizar los procesos del CEISH-ESPOCH para reducir tiempos de evaluación, garantizar trazabilidad y cumplir con PET 2023 y normativa MSP
Alcance Funcional	Gestión de protocolos (IO/EI/EC), validación documental, evaluación ética, seguimiento, notificaciones, auditoría, reportes MSP
Requisitos Clave	RF-001 a RF-018 (funcionales) + RNF-001 a RNF-030 (no funcionales)
Metas de Calidad	Mantenibilidad (ISO 25010), Seguridad JWT+RBAC, Auditoría inmutable 7 años, Performance <3s transacciones
Stakeholders Principales	Secretaria CEISH (Karina Albuja), Presidenta (Dra. Verónica Delgado), Evaluadores, Investigadores ESPOCH, ARCSA/DIS

1.2 Stakeholders

Rol/Nombre	Contacto	Expectativas de la Arquitectura
Secretaria CEISH	Karina Albuja	Validación rápida de documentos, notificaciones automáticas, reducción de carga manual en 65%
Presidenta del Comité	Dra. Verónica Delgado	Dashboard de carga de evaluadores, asignación equilibrada, generación automática de resoluciones
Evaluadores	Comité CEISH (8 miembros)	Acceso remoto a protocolos, formularios digitales, plazos claros (8/15 días)
Investigadores	Estudiantes/Docentes ESPOCH	Formulario dinámico por tipo de estudio, estado visible en tiempo real, carga masiva de documentos
ARCSA/DIS	Entidades externas	Notificación automática de EAG/RAGI en 48h, formatos Anexo 20/21 compatibles
Desarrolladores	Kevin Quilligana, Liliana Paguay	Código testeable, documentación clara, patrones aplicados consistentemente

Docente Evaluador	Ing. Jaime Camacho	Cumplimiento de estándares académicos, trazabilidad requisitos→arquitectura→código
--------------------------	--------------------	--

2 Restricciones de Arquitectura

2.1 Restricciones Técnicas

Restricción	Justificación	Impacto en Diseño
Angular 16+	Definido en perfil de proyecto	Frontend component-based con MVC pattern
NestJS + Node.js	Definido en perfil de proyecto	Backend modular con inyección de dependencias
PostgreSQL 15+	Definido en perfil de proyecto	TypeORM con Reflection pattern para persistencia
JWT + Refresh Tokens	Seguridad requerida (RNF-002)	AuthService Singleton + Guards en NestJS
Docker + Docker Compose	Despliegue consistente	Contenedores para frontend, backend, DB

2.2 Restricciones Organizacionales

Restricción	Justificación	Impacto en Diseño
Equipo: 2 desarrolladores	Recursos académicos limitados	Arquitectura viable, sin over-engineering
Tiempo: 5 meses (11 sprints)	Periodo académico	Priorización estricta, MVP primero
Metodología: Scrum	Estándar del proyecto	Historias de usuario → Product Backlog → Sprints
Validación continua con PO	Karina Albuja como Product Owner	Entregas incrementales, feedback temprano

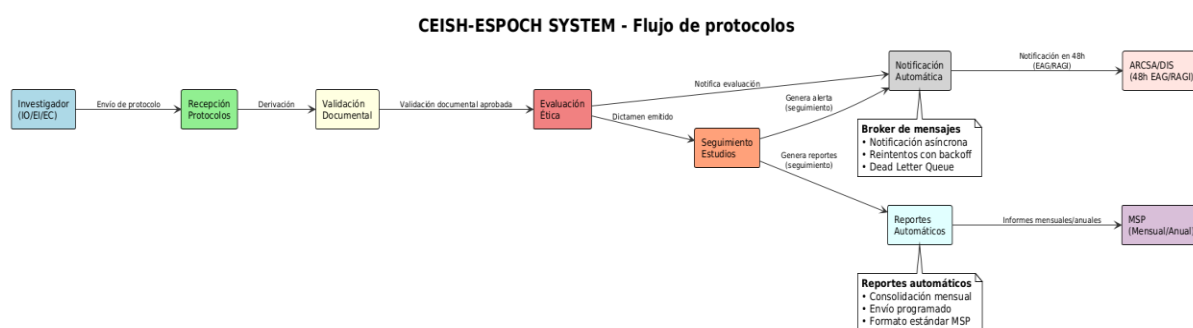
2.3 Restricciones Normativas

Restricción	Fuente	Impacto en Diseño
PET CEISH-ESPOCH V2 (2023)	Documento normativo interno	Lógica de negocio en Domain layer, validaciones por tipo de estudio
Acuerdo Ministerial 00015-2021	MSP Ecuador	Requisitos documentales para IO/EI en formularios dinámicos

Acuerdo Ministerial 00005-2022	MSP Ecuador	Requisitos específicos para ensayos clínicos (EC)
Ley Orgánica de Protección de Datos	Ecuador	Encriptación AES-256, acceso restringido por roles
Retención 7 años (PET 5.1.b)	PET CEISH- ESPOCH	Auditoría inmutable, logs no modificables

3 Contexto y Alcance

3.1 Contexto de Negocio



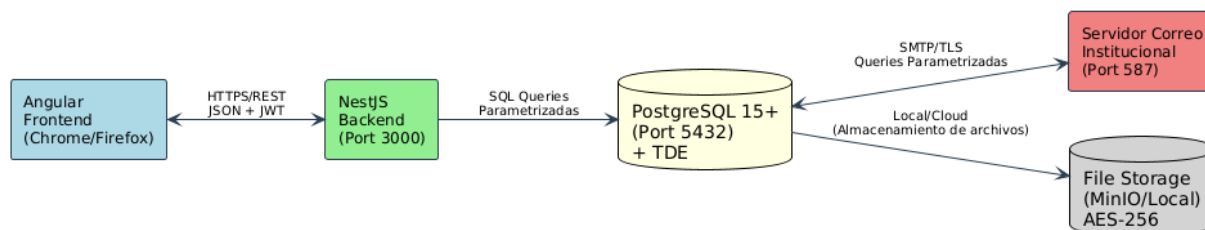
Interfaces de Dominio

Actor	Entrada al Sistema	Salida del Sistema
Investigador	Protocolo (Anexos 1-6 según tipo), Documentos PDF	Código CEISH-ESPOCH-XXX-###-AAAA, Estado, Observaciones
Secretaria	Validación checklist, Asignación de evaluadores	Constancia de recepción, Notificaciones, Reportes
Evaluador	Informe de evaluación (Anexos 9-11), Dictamen	Resultado: Aprobado/Condicionado/No Aprobado
Presidenta	Asignación de evaluadores, Resolución final	Carta de aprobación (Anexos 13-16), Vigencia 1 año
ARCSA/DIS	- (receptor pasivo)	Notificación EAG/RAGI (Anexos 20-21) en 48h

MSP	- (receptor pasivo)	Reportes mensuales/anuales (Excel/PDF)
------------	---------------------	--

3.2 Contexto Técnico

INFRAESTRUCTURA TÉCNICA - CEISH-ESPOCH



Mapeo Interfaces Técnicas

Canal	Protocolo	Datos	Frecuencia
Frontend ↔ Backend	HTTP/HTTPS + REST API	JSON	Por interacción usuario
Backend ↔ Database	TCP/IP + PostgreSQL Protocol	SQL parametrizado	Continuo
Backend ↔ SMTP	SMTP + TLS (puerto 587)	MIME	Por evento (aprobación, vencimiento)
Backend ↔ File Storage	Local API / S3-compatible	PDF/DOCX encriptados	Por subida de documento
Backend ↔ ARCSA/DIS	HTTPS + JSON (futuro)	Formato Anexo 20/21	Por EAG/RAGI reportado

4 Estrategia de Solución

4.1 Decisiones Fundamentales

Decisión	Alternativas Consideradas	Criterios de Selección	Justificación
Estilo: CAPAS + CLEAN/HEXAGONAL	Monolito MVC tradicional, Microservicios, Event-Driven	Mantenibilidad, Testabilidad, Viabilidad para 2 devs en 5 meses	Separa Domain layer de infraestructura, facilita testing, viable para restricciones del proyecto
Factory Method para protocolos	Switch-case en controller, Abstract Factory	Extensibilidad, Open/Closed Principle	Nuevos tipos de estudio (IO/EI/EC) sin modificar código

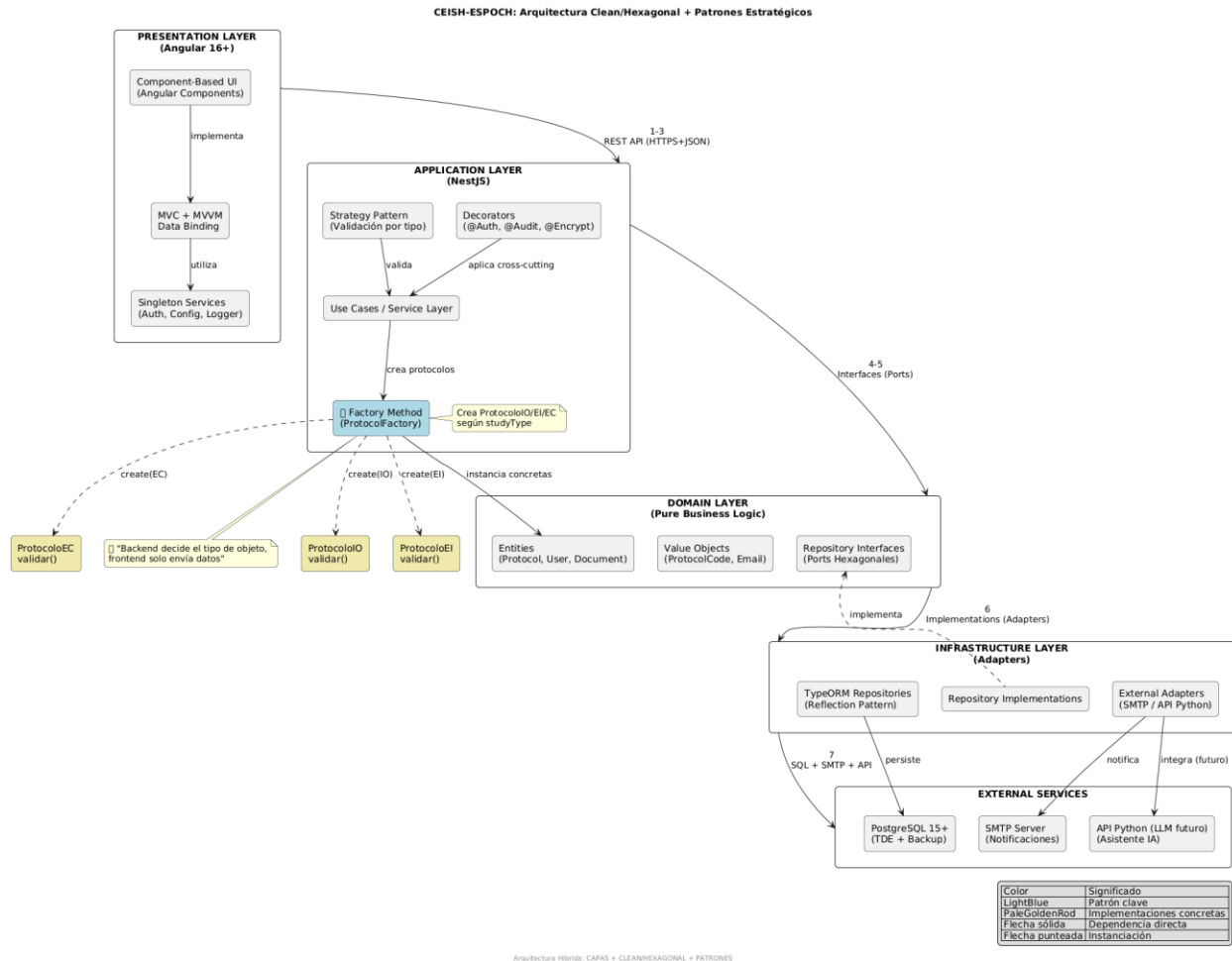
			existente; alineado con PET 4.1.2
Strategy Pattern para validaciones	Validación monolítica, Rules Engine externo	Flexibilidad, Mantenibilidad	Validaciones diferentes por tipo de estudio; fácil añadir nuevas reglas PET
Decorator para @Audit/@Encrypt	AOP externo, Logging manual	Transparencia, Reusabilidad	Cross-cutting concerns aplicables globalmente sin modificar lógica de negocio
Broker Pattern para ARCSA/DIS	Llamada HTTP directa, Cola de mensajes	Resiliencia, Testabilidad	Aislamiento de APIs externas, mocks para testing, reintentos automáticos
Singleton para AuthService	Instanciación múltiple, Dependency Injection manual	Consistencia, Eficiencia	Única instancia para gestión JWT en toda la aplicación
TypeORM + Reflection	Query builder manual, Stored procedures	Productividad, Portabilidad	Mapeo dinámico entidad↔tabla, agnóstico a cambios de esquema

4.2 Enfoque para Metas de Calidad

Meta de Calidad	Estrategia Arquitectónica	Patrón/Técnica
Mantenibilidad	Separación clara de responsabilidades por capas	Clean Architecture + Hexagonal Ports-Adapters
Testabilidad	Domain layer sin dependencias externas	Dependency Injection + Repository Interfaces mockeables
Seguridad	Autenticación centralizada + encriptación transparente	Singleton AuthService + Decorator @Encrypt
Auditabilidad	Logs inmutables de todas las acciones críticas	Decorator @Audit + Tabla audit_log (solo INSERT)
Extensibilidad	Nuevos tipos de estudio sin modificar código existente	Factory Method + Strategy Pattern
Resiliencia	Notificaciones externas con reintentos y fallback	Broker Pattern + Retry Policy + Mock Adapter
Performance	Consultas optimizadas + caching estratégico	TypeORM indexes + Decorator @Cache + CQRS-lite

5 Vista de Bloques de Construcción

5.1 Nivel 1: Sistema Completo (White Box)



Motivación de la Descomposición

- Aislamiento de lógica de negocio:** Domain layer no conoce infraestructura → fácil cambiar tecnología sin afectar reglas PET
- Testabilidad:** Use Cases testables con mocks de Repository Interfaces

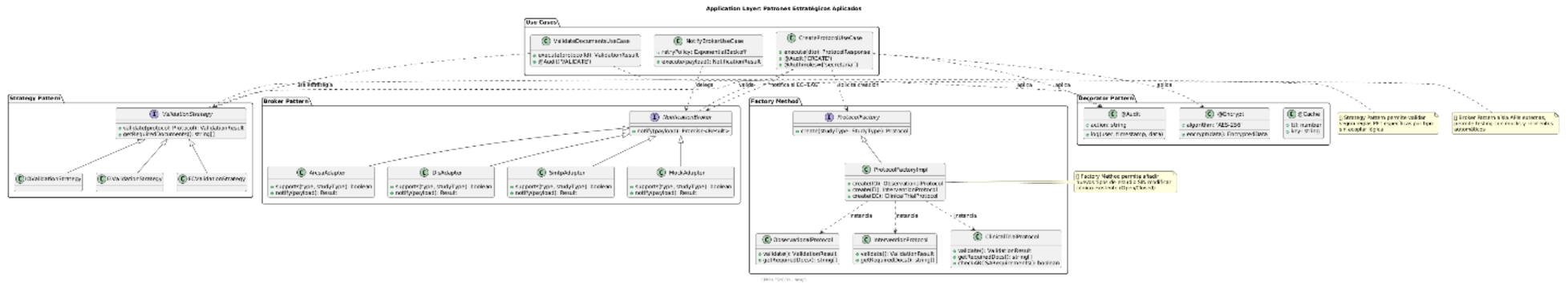
3. **Extensibilidad:** Nuevos tipos de estudio vía Factory + Strategy sin modificar código existente

4. **Mantenibilidad:** Cross-cutting concerns (@Audit, @Encrypt) aplicados vía Decorator sin duplicación

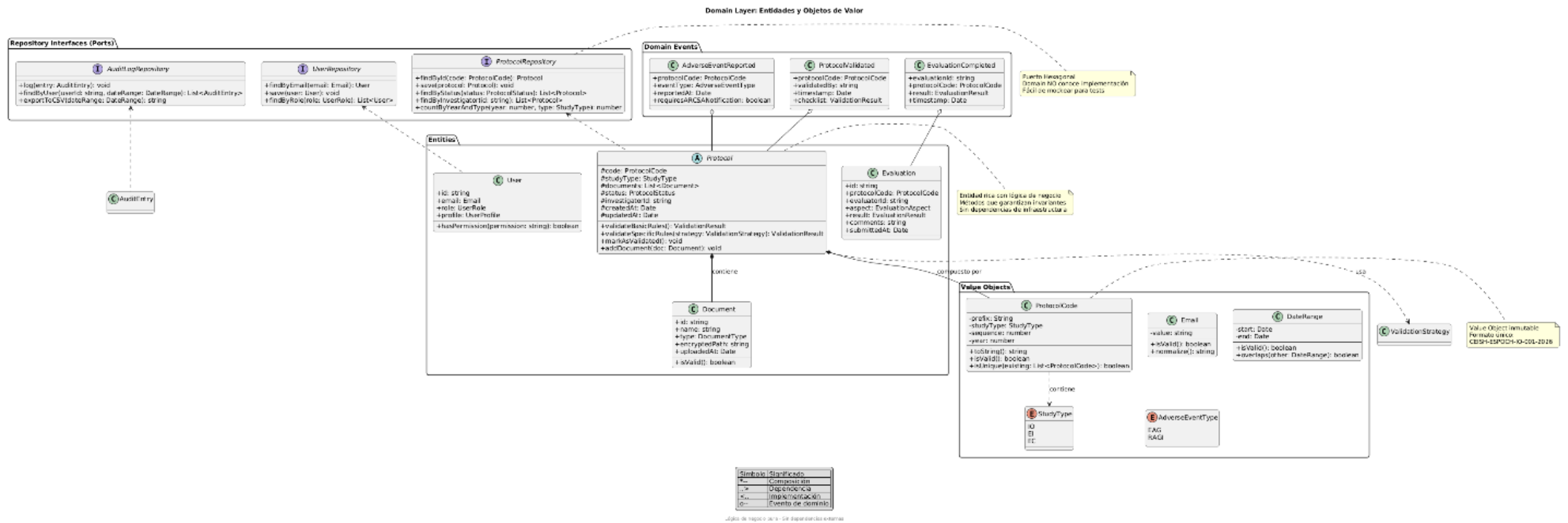
Bloques Contenidos (Black Boxes)

Nombre	Responsabilidad	Interfaces Clave
ProtocolModule (Angular)	Formularios dinámicos IO/EI/EC, carga de documentos, estado visible	ProtocolService.create(), ProtocolService.getStatus()
ProtocolUseCases (NestJS)	Orquestación de casos de uso: crear, validar, asignar, evaluar	CreateProtocolUseCase.execute(), ValidateDocumentsUseCase.execute()
ProtocolFactory (NestJS)	Creación de protocolos según tipo de estudio (IO/EI/EC)	ProtocolFactory.create(studyType): Protocol
ValidationStrategy (NestJS)	Validaciones específicas por tipo de estudio según PET	ValidationStrategy.validate(protocol): ValidationResult
NotificationBroker (NestJS)	Coordinación de notificaciones a ARCSA/DIS/SMTP	NotificationBroker.notify(payload): Promise<Result>
ProtocolRepository (Interface)	Contrato para persistencia de protocolos (Port Hexagonal)	findById(), save(), findByStatus()
TypeORM ProtocolRepository (Impl)	Implementación concreta con TypeORM + PostgreSQL (Adapter)	Mapeo entidad↔tabla vía Reflection pattern
@Audit Decorator (NestJS)	Registro automático de logs para acciones críticas	Aplicación transparente en métodos de Use Cases
AuthService (Singleton)	Gestión centralizada de JWT + Refresh Tokens	login(), validateToken(), refreshToken()

5.2 Nivel 2: Application Layer (White Box)

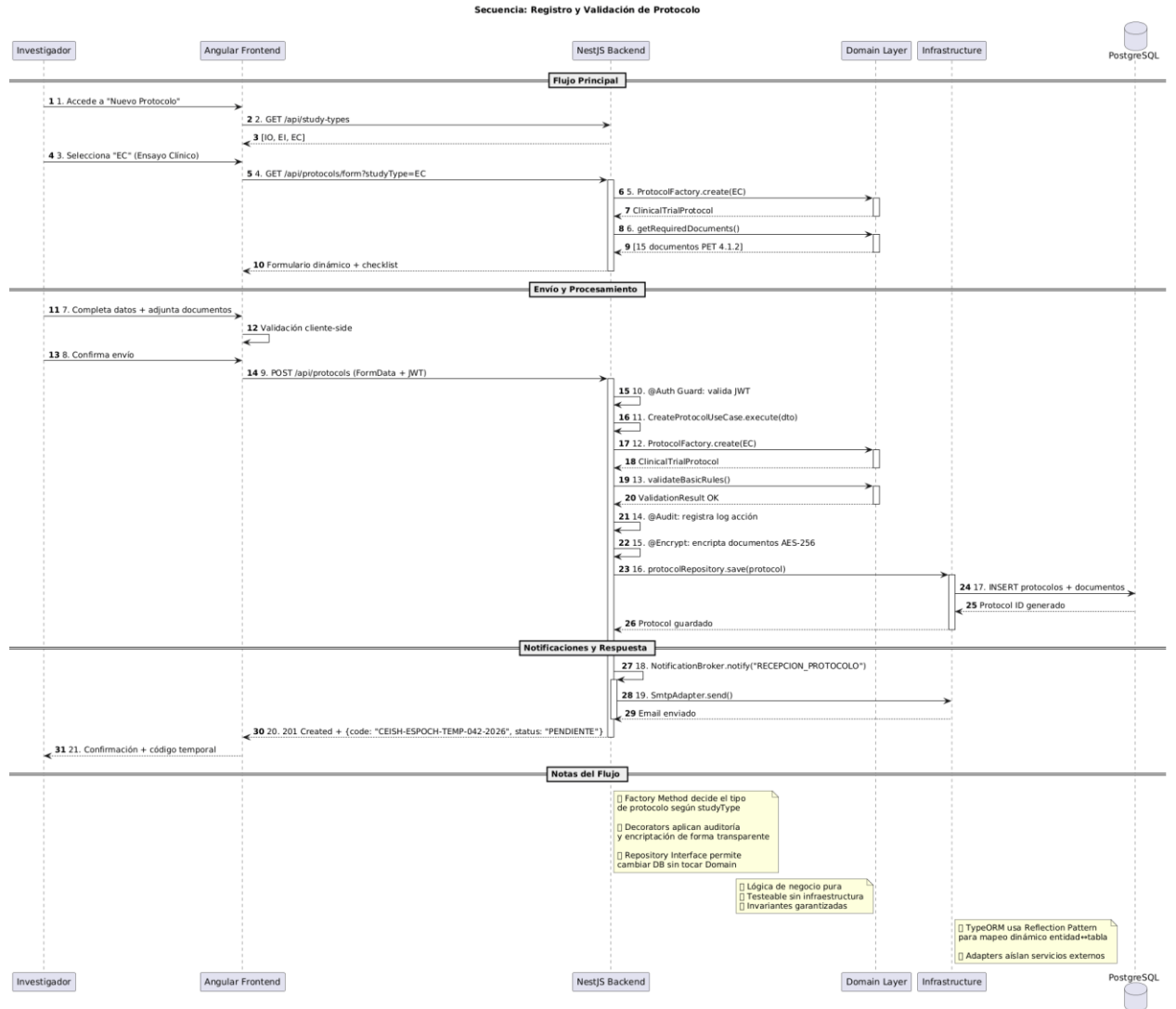


5.3 Nivel 3: Domain Layer Entities (White Box)



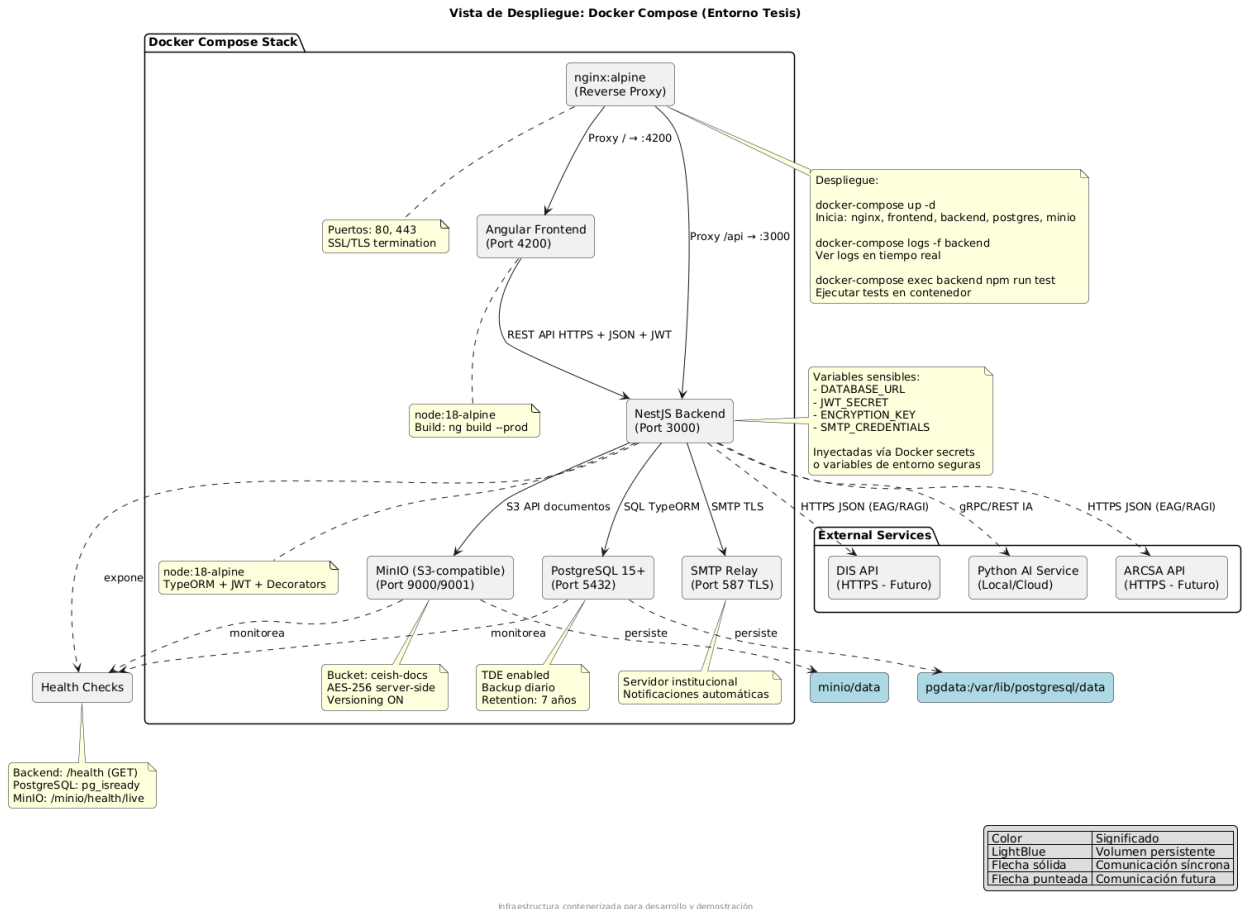
6 Vista de Tiempo de Ejecución

6.1 Escenario: Registro y Validación de Protocolo



7 Vista de Despliegue

7.1 Infraestructura Nivel 1: Entorno de Desarrollo/Tesis



7.2 Mapeo de Bloques a Infraestructura

Bloque de Software	Elemento de Infraestructura	Justificación
Angular Frontend	Contenedor Docker + nginx	Aislamiento, escalabilidad horizontal futura
NestJS Backend	Contenedor Docker + Node.js 18	Consistencia de entorno, fácil despliegue
ProtocolRepository	PostgreSQL 15 + TypeORM	ACID, TDE para encriptación en reposo, TypeORM para Reflection pattern
Document Storage	MinIO (S3-compatible)	Escalable, versioning automático, compatible con AWS S3 si migra a cloud
Email Notifications	SMTP Institucional (puerto 587 TLS)	Cumplimiento institucional, sin costos adicionales

Audit Logs	Tabla audit_log en PostgreSQL (sólo INSERT)	Inmutabilidad garantizada por permisos de BD, retención 7 años vía backup
JWT Auth	Memoria + Redis (opcional para refresh tokens)	Stateless para escalabilidad, Redis para invalidación de tokens si es necesario

8 Conceptos Transversales

8.1 Estrategia de Testing

Capa	Tipo de Prueba	Herramienta	Cobertura Objetivo
Domain	Unitarias (sin dependencias)	Jest	100% entities, value objects, domain services
Application	Unitarias con mocks	Jest + @golevelup/ts-jest	90% use cases, factories, strategies
Infrastructure	Integración con DB real	Jest + testcontainers	80% repositories, adapters, decorators
Presentation	E2E + Componentes	Cypress + Jest Testing Library	70% flujos críticos (registro, validación, notificación)
Security	Penetration testing básico	OWASP ZAP + manual	Validar JWT, RBAC, inyección SQL, XSS

9 Decisiones Arquitectónicas

9.1 Registro de Decisiones Clave (ADR Format)

9.1.1 ADR-001: Estilo Arquitectónico Híbrido

Fecha: 15-abr-2026

Estado: Aceptado

Contexto: Proyecto académico con 2 desarrolladores, 5 meses, requisitos PET complejos, necesidad de mantenibilidad y testabilidad.

Decisión: Adoptar arquitectura híbrida: **CAPAS/LAYERS + CLEAN/HEXAGONAL + PATRONES ESTRATÉGICOS.**

Alternativas Consideradas:

- Monolito MVC tradicional: Difícil testabilidad, acoplamiento alto, evolución costosa

- Microservicios: Over-engineering, complejidad de orquestación, inviable para 2 devs
- Event-Driven puro: Debugging complejo, consistencia eventual problemática para auditoría

Consecuencias Positivas:

- Domain layer aislado → fácil cambiar infraestructura sin tocar lógica PET
- Use Cases testables con mocks de Repository Interfaces
- Factory + Strategy permiten extender tipos de estudio sin modificar código existente
- Decorator aplica @Audit/@Encrypt de forma transparente

Consecuencias Negativas:

- Curva de aprendizaje inicial para Clean/Hexagonal
- Mayor boilerplate que MVC tradicional
- Requiere disciplina para mantener capas separadas

Justificación: Cumple restricciones de tiempo/equipo mientras garantiza calidad arquitectónica para defensa de tesis y posible evolución post-tesis.

9.1.2 ADR-002: Factory Method para Creación de Protocolos

Fecha: 18-abr-2026

Estado: Aceptado

Contexto: PET 4.1.2 define checklists diferentes para IO (9 docs), EI (11 docs), EC (15 docs). Nuevos tipos de estudio podrían añadirse en el futuro.

Decisión: Implementar **ProtocolFactory** con Factory Method pattern para crear instancias de Protocol según studyType.

Alternativas Consideradas:

- Switch-case en controller: Acoplamiento alto, violación Open/Closed
- Abstract Factory: Overkill para 3 tipos iniciales, complejidad innecesaria

Consecuencias Positivas:

- Nuevo tipo de estudio = nueva clase + registro en Factory (sin modificar código existente)
- Validaciones específicas encapsuladas en cada implementación concreta
- Testing aislado por tipo de protocolo

Consecuencias Negativas:

- Ligera sobrecarga de clases para solo 3 tipos iniciales
- Requiere documentación clara del contrato Protocol

Justificación: Alineado con PET 4.1.2 y principio Open/Closed; facilita mantenimiento y evolución.

9.1.3 ADR-003: Decorator Pattern para Auditoría y Encriptación

Fecha: 22-abr-2026

Estado: Aceptado

Contexto: Requisitos RNF-002 (seguridad JWT), RNF-009 (encriptación documentos), PET 5.1.b (auditoría inmutable 7 años) son cross-cutting concerns que aplican a múltiples Use Cases.

Decisión: Implementar **@Audit** y **@Encrypt** como decorators en NestJS para aplicar funcionalidad transversal sin modificar lógica de negocio.

Alternativas Consideradas:

- AOP externo (NestJS Interceptors genéricos): Menos explícito, difícil de rastrear
- Logging manual en cada método: Duplicación, propenso a omisiones
- Blockchain para auditoría: Over-engineering, PostgreSQL + triggers + hashes cumple PET 5.1.b

Consecuencias Positivas

- Auditoría automática en métodos marcados con **@Audit**
- Encriptación transparente con **@Encrypt** sin modificar código de negocio

- Fácil añadir nuevos decorators (@Cache, @RateLimit) sin afectar existentes

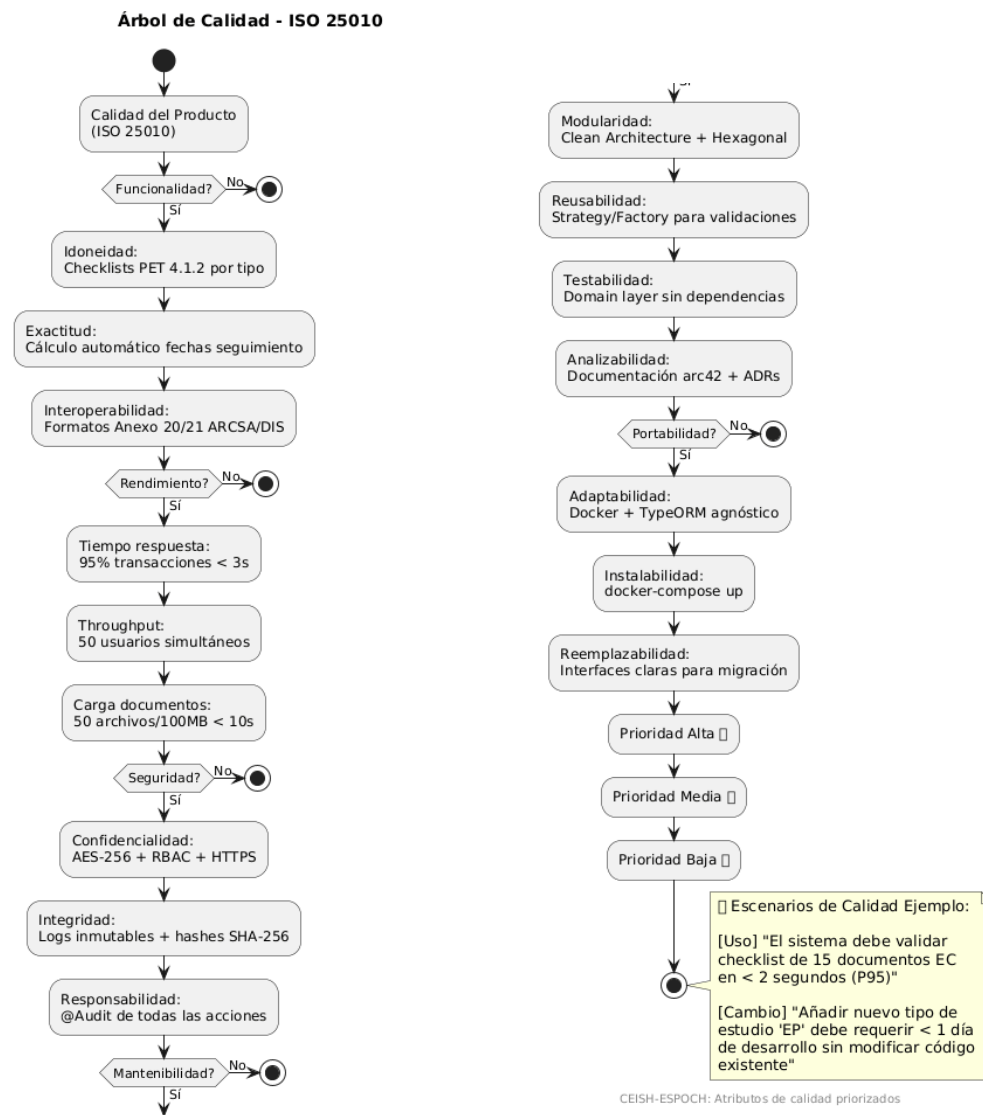
Consecuencias Negativas

- Debugging ligeramente más complejo (stack trace con decorators)
- Requiere convención de equipo para usar decorators consistentemente

Justificación: Maximiza reusabilidad y mantenibilidad para cross-cutting concerns críticos de seguridad y auditoría.

10 Requisitos de Calidad

10.1 Árbol de Calidad (ISO 25010)



10.2 Escenarios de Calidad Detallados

Escenario de Uso: Tiempo de Respuesta en Validación Documental

Elemento	Descripción
Contexto	Secretaria valida checklist de 15 documentos para protocolo EC
Estímulo	Secretaria hace clic en "Validar Documentos" en dashboard
Artefacto	ValidateDocumentsUseCase + TypeORM Repository
Respuesta	Sistema retorna resultado en < 2 segundos (95% de casos)
Medida de Respuesta	Tiempo desde clic hasta renderizado de resultado (frontend performance API)
Criterio de Aceptación	P95 < 2s, P99 < 5s, medido con Lighthouse + backend logs

Escenario de Cambio: Añadir Nuevo Tipo de Estudio (Ej: "Estudio Piloto")

Elemento	Descripción
Contexto	PET actualiza y añade tipo de estudio "EP" (Estudio Piloto) con checklist de 7 documentos
Estímulo	Product Owner solicita soporte para nuevo tipo de estudio sin downtime
Artefacto	ProtocolFactory + ValidationStrategy + Frontend FormFactory
Respuesta	Nuevo tipo disponible en producción en < 1 día de desarrollo
Medida de Respuesta	Tiempo desde merge a producción + pruebas de regresión
Criterio de Aceptación	- Nuevas clases: PilotProtocol + PilotValidationStrategy < 200 líneas c/u - Factory registra nuevo tipo sin modificar código existente - Tests de integración pasan sin modificar casos existentes - Frontend renderiza formulario dinámico sin cambios en componentes base

11 Riesgos y Deuda Técnica

11.1 Riesgos Identificados

Riesgo	Probabilidad	Impacto	Mitigación	Owner
Curva de aprendizaje Clean/Hexagonal	Media	Medio	- Documentación arc42 detallada - Pair programming inicial - ADRs para decisiones clave	Kevin/Liliana
Complejidad inicial mayor que MVC tradicional	Alta	Bajo	- Iniciar con Level 1 (módulos), evolucionar a Level 2/3 - Buffer de 20% en estimación de sprints	Kevin/Liliana
Integración con ARCSA/DIS no disponible durante tesis	Alta	Medio	- MockAdapter para testing - Adapter pattern permite cambiar implementación post-tesis - Notificación vía email como fallback	Kevin
Cambios en PET durante desarrollo	Baja	Alto	- Strategy Pattern para validaciones intercambiables - Parámetros PET configurables en BD - Validación continua con PO (Karina)	Liliana
Performance con 10k+ protocolos en BD	Baja	Medio	- Indexación estratégica en PostgreSQL - CQRS-lite para dashboard (lecturas optimizadas) - Decorator @Cache para consultas frecuentes	Kevin
Seguridad: JWT compromise o inyección SQL	Baja	Alto	- Refresh tokens + expiración 24h - Queries parametrizadas TypeORM - Sanitización de inputs + Helmet.js - Pen-testing básico pre-defensa	Kevin

11.2 Deuda Técnica Consciente

Deuda	Justificación	Plan de Pago	Prioridad
MockAdapter para ARCSA/DIS	APIs externas no disponibles durante periodo académico	Implementar ArcsaAdapter/DisAdapter reales post-tesis cuando APIs estén disponibles	Media
Asistente basado en IA en plantillas (no LLM)	Costos/privacidad de APIs externas de IA	Evaluar integración con LLM institucional post-tesis si hay presupuesto	Baja
Despliegue local para tesis (no producción institucional)	Restricciones de TI-ESPOCH para proyectos de estudiantes	Migrar a infraestructura institucional post-aprobación de tesis	Alta
Cobertura de tests 80% (no 100%)	Balance tiempo/calidad para periodo académico	Incrementar a 90% en sprints finales para módulos críticos	Media

12 Glosario

Término	Definición
CEISH	Comité de Ética de Investigación en Seres Humanos (ESPOCH)
PET	Procesos Estandarizados de Trabajo (documento normativo CEISH-ESPOCH V2, 2023)
IO/EI/EC	Investigación Observacional / Estudio de Intervención / Ensayo Clínico (tipos de estudio)
ProtocolCode	Value Object: Código único CEISH-ESPOCH-XXX-###-AAAA (XXX=IO/EI/EC, ###=secuencial, AAAA=año)
EAG/RAGI	Evento Adverso Grave / Reacción Adversa Grave Inesperada (reportables a ARCSA/DIS en 48h)
ARCSA	Agencia Nacional de Regulación, Control y Vigilancia Sanitaria (Ecuador)
DIS	Dirección Nacional de Inteligencia de la Salud (MSP Ecuador)
Clean Architecture	Estilo arquitectónico de Robert C. Martin: separación por capas concéntricas con inversión de dependencias
Hexagonal Architecture	Patrón Ports-Adapters de Alistair Cockburn: aislamiento de lógica de negocio mediante interfaces

Factory Method	Patrón de creación: delega la instanciación de objetos concretos a subclases (AS 18)
Strategy Pattern	Patrón de comportamiento: encapsula algoritmos intercambiables (validaciones por tipo de estudio)
Decorator Pattern	Patrón de estructura: añade funcionalidad dinámica a objetos sin modificar su código (@Audit, @Encrypt)
Broker Pattern	Patrón para sistemas distribuidos: intermediario que coordina comunicación entre componentes (AS 13)
Singleton Pattern	Patrón de creación: garantiza única instancia de una clase con acceso global (AuthService)
Reflection/ORM	Patrón de persistencia: mapeo dinámico entidad↔tabla vía metadatos (TypeORM)
CQRS-lite	Variante simplificada de Command Query Responsibility Segregation: separación lecturas/escrituras para optimización
ADR	Architecture Decision Record: documento que captura decisión arquitectónica clave con contexto y consecuencias